

Pandas

Data Transformation: Grouping, Lambda, Pivoting, Pipelines

DAEN 300

Alexander Abuabara

Fall 2026



Outline

Setup and Inspection

Selecting and Filtering

GroupBy

Pivoting and Reshaping

String Operations

Chaining and Performance

Cheat Sheet and Wrap-up

Assignment

"Dataset" for Examples

```
import pandas as pd
# load pandas with the standard alias pd

import numpy as np
# load NumPy

df = pd.DataFrame({
    'region': ['East', 'East', 'West', 'West', 'East'],
    'category': ['A', 'B', 'A', 'B', 'A'],
    'store': ['S1', 'S1', 'S2', 'S2', 'S3'],
    'price': [10, 20, 30, 40, 50],
    'qty': [1, 2, 3, 4, 5],
    'name': [' ada lovelace ', 'Grace Hopper', None,
            'Alan Turing', 'Katherine Johnson']
})
```

Pandas

- ▶ Work with labeled tables instead of raw arrays.
- ▶ Clean, reshape, join, aggregate, and analyze data quickly.
- ▶ Use vectorized operations whenever possible for speed and clarity.

```
s = df['price']  
# a Series; one labeled column from the table  
  
small = df[['region', 'price']]  
# a DataFrame with two selected columns
```

Inspection

```
df.head()
```

```
# returns the first 5 rows; useful first look
```

```
df.info()
```

```
# shows columns, dtypes, memory, and non-null counts
```

```
df.describe()
```

```
# price mean = 30.0, qty mean = 3.0 in the toy data
```

```
df.shape
```

```
# (5, 6), meaning 5 rows and 6 columns
```

```
df.sample(2, random_state=1)
```

```
# two random rows; avoids only seeing the top of the file
```

```
df.dtypes
```

```
# price and qty are int64; name is object/string-like
```

Selection: loc vs iloc

```
df.loc[0, 'price']  
# 10; label-based row/column lookup  
  
df.iloc[0, 3]  
# 10; position-based lookup, row 0 and column 3  
  
df.loc[df['price'] > 25, ['region', 'price']]  
# rows with prices 30, 40, and 50  
  
mask = (df['price'] > 25) & (df['category'] == 'A')  
# True for the West-A row and East-A row with price 50  
  
df[mask]  
# keeps only rows where the mask is True
```

Use & and |, not AND and OR; wrap each condition in parentheses.

Readable Filters with query

```
df.query('price > 25 and category == "A"')  
# returns rows with price 30 and 50 in category A  
  
threshold = 25  
# create a Python variable for the cutoff  
  
df.query('price > @threshold')  
# @threshold injects the Python variable into the query
```

- ▶ Good for long filters and method chains.
- ▶ Column names with spaces require backticks, for example 'unit price'.

GroupBy Basics

```
df.groupby('category')['price'].mean()
# A -> 30.0, B -> 30.0

df.groupby('category').agg(
    avg_price=('price', 'mean'),
    orders=('price', 'size'),
    max_price=('price', 'max')
)
# clean columns named avg_price, orders, max_price
```

Named aggregation avoids messy MultiIndex column names.

GroupBy: Multiple Columns and Functions

```
df.groupby(['region', 'category'])['price'].sum()
# East/A -> 60, East/B -> 20, West/A -> 30, West/B -> 40

df.groupby(['region', 'category']).agg(
    total_sales=('price', 'sum'),
    total_qty=('qty', 'sum')
)
# one row per region-category combination

df.groupby('category')['price'].agg(['mean', 'min', 'max'])
# A has min 10 and max 50; B has min 20 and max 40
```

Lambda

```
lambda x: x * 2
# an anonymous function; input x, output x * 2

(lambda x: x * 2)(5)
# 10

def double(x):
    return x * 2
# same logic as lambda x: x * 2, but with a name
```

- ▶ Syntax: `lambda argument: expression`.
- ▶ A lambda has exactly one expression, not a full block of statements.
- ▶ In pandas, the argument is often a row, Series, group, or DataFrame.

Lambda in GroupBy

```
df.groupby('category')['price'].apply(lambda s: s.max() - s.min())  
# A -> 40, B -> 20; range of prices by category
```

```
df.groupby('category').filter(lambda g: len(g) >= 2)  
# keeps A and B because each category has at least 2 rows
```

```
df.groupby('category')['price'].transform(lambda s: s - s.mean())  
# subtracts each category mean from each row's price
```

Interpretation: *s* is one category's price Series; *g* is one category's DataFrame.

GroupBy Tricks: transform and filter

```
df['cat_total'] = df.groupby('category')['price'].transform('sum')  
# category A rows receive 90; category B rows receive 60  
  
df['pct_of_cat'] = df['price'] / df['cat_total']  
# first row is 10 / 90 = 0.111...  
  
df.groupby('category').filter(lambda g: g['price'].mean() >= 30)  
# keeps categories whose average price is at least 30
```

transform returns one value per original row, so it is ideal for adding group-level columns.

Top Values per Group

```
df['category'].value_counts(normalize=True)
# A -> 0.60, B -> 0.40

df.groupby('category')['price'].apply(lambda s: s.nlargest(2))
# top two prices inside each category

df.sort_values('price').groupby('category').tail(1)
# max-price row for A is 50; max-price row for B is 40
```

Pivot Tables

```
pd.pivot_table(  
    df,  
    index='region',  
    columns='category',  
    values='price',  
    aggfunc='mean'  
)  
# rows are East/West; columns are A/B; cells are mean price
```

- ▶ index: rows in the result.
- ▶ columns: values spread across columns.
- ▶ aggfunc: how duplicate cells are summarized.

Pivot Tables: Practical Options

```
pd.pivot_table(  
    df,  
    index=['region', 'store'],  
    columns='category',  
    values=['price', 'qty'],  
    aggfunc={'price': 'mean', 'qty': 'sum'},  
    margins=True,  
    fill_value=0  
)  
# adds All totals and replaces missing cells with 0
```

`margins=True` gives row and column totals without a separate calc.

pivot, pivot_table, crosstab

```
df.pivot(index='store', columns='category', values='price')  
# reshapes only if each store-category pair is unique  
  
pd.crosstab(df['category'], df['region'])  
# frequency table: category by region  
  
pd.crosstab(df['category'], df['region'], normalize='index')  
# row percentages that sum to 1.0 within each category
```


Reshaping: melt, stack, unstack

```
wide = pd.DataFrame({'id': [1, 2], 'jan': [10, 20], 'feb': [30, 40]})  
# sales are stored across month columns  
  
pd.melt(wide, id_vars='id', value_vars=['jan', 'feb'],  
        var_name='month', value_name='sales')  
# converts jan/feb columns into month and sales rows  
  
df.set_index(['region', 'category'])['price'].unstack('category')  
# category values become columns A and B
```

.str

```
df['name'].str.lower()
# 'Grace Hopper' becomes 'grace hopper'

df['name'].str.strip()
# ' ada lovelace ' becomes 'ada lovelace'

df['name'].str.contains('hopper', case=False, na=False)
# True for Grace Hopper; False for missing names

df['name'].str.replace(r'\s+', ' ', regex=True)
# collapses repeated spaces into one space

df['name'].str.len()
# counts characters, including leading/trailing spaces
```

String Tricks: Split and Extract

```
clean_name = df['name'].str.strip()
# remove extra spaces before splitting

df[['first', 'last']] = clean_name.str.split(' ', n=1, expand=True)
# 'Ada Lovelace' -> first='Ada', last='Lovelace'

df['first_initial'] = df['first'].str.extract(r'^(\w)')
# Ada -> A, Grace -> G

df['words'] = clean_name.str.count(r'\w+')
# 'Katherine Johnson' has 2 words
```

String Tricks: Join, Pad, Case

```
df['full'] = df['first'].str.cat(df['last'], sep=' ')  
# first='Grace', last='Hopper' -> 'Grace Hopper'  
  
df['code'] = df.index.astype(str).str.zfill(3)  
# index 7 would become '007'  
  
df['title'] = df['name'].str.strip().str.title()  
# 'ada lovelace' -> 'Ada Lovelace'  
  
df['slug'] = df['title'].str.lower().str.replace(r'\W+', '-', regex=  
    True)  
# 'Grace Hopper' -> 'grace-hopper'
```

Categorical Dtype for Repeated Strings

```
df['category'] = df['category'].astype('category')  
# stores repeated A/B values more compactly than object strings  
  
df['size'] = pd.Categorical(  
    ['S','M','L','M','S'],  
    categories=['S','M','L','XL'],  
    ordered=True  
)  
# sorting respects S < M < L < XL
```

- ▶ Useful for low-cardinality text columns.
- ▶ Can improve memory use and some grouping/sorting operations.

Comparing Object Sizes

```
import sys
import numpy as np

vector_a = [1, 2, 3, 4, 5]
array_b = np.array([1, 2, 3, 4, 5])

sys.getsizeof(vector_a) # list object
sys.getsizeof(array_b) # NumPy array object
array_b.nbytes # raw array data
```

- ▶ `sys.getsizeof()` measures the Python object itself.
- ▶ `vector_a` is a list of references to Python objects.
- ▶ `array_b` stores values in a compact contiguous data block.
- ▶ `array_b.nbytes` reports only the raw numeric data size.

Object Size vs. Data Size

Object size includes the container/metadata overhead,
while data size measures only the memory occupied by the stored values.

```
array_b = np.array([1, 2, 3, 4, 5])  
  
array_b.size # number of elements  
array_b.itemsize # bytes per element  
array_b.nbytes # size * itemsize
```

$$\text{array_b.nbytes} = \text{array_b.size} \times \text{array_b.itemsize}$$

- ▶ `sys.getsizeof(array_b)` includes array-object overhead.
- ▶ `array_b.nbytes` includes only the data buffer.
- ▶ Metadata such as shape, dtype, and strides is not included in `nbytes`.

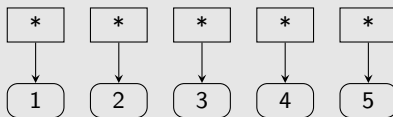
Why Lists Do Not Have nbytes

```
vector_a = [1, 2, 3, 4, 5]
```

```
sys.getsizeof(vector_a) # list container + references
```

```
vector_a.nbytes # AttributeError
```

vector_a list container



separate Python integer objects

- ▶ `nbytes` is specific to NumPy arrays.
- ▶ Lists store references, not compact raw values.
- ▶ `sys.getsizeof(vector_a)` is not recursive ... it measures only the memory used directly by the list container, not the memory used by the objects referenced inside the list.

Method Chaining with assign

```
result = (  
    df  
    .query('price > 0')  
    # keep valid positive prices  
    .assign(  
        revenue=lambda d: d['price'] * d['qty'],  
        # lambda d sees the current DataFrame in the chain  
        price_band=lambda d: np.where(d['price'] >= 30, 'high', 'low')  
        # price 10/20 -> low; price 30/40/50 -> high  
    )  
    .groupby('price_band')['revenue'].sum()  
    # total revenue by price band  
    .sort_values(ascending=False)  
    # largest revenue band first  
)
```

Performance: Vectorize, Do Not Loop

```
df['total_slow'] = df.apply(lambda r: r['price'] * r['qty'], axis=1)
# works, but calls Python once per row; slow on large data

df['total_fast'] = df['price'] * df['qty']
# same result, vectorized and much faster

df['flag'] = np.where(df['price'] > 30, 'high', 'low')
# prices 40 and 50 become high

df['tier'] = pd.cut(df['price'], bins=[0, 20, 40, np.inf],
                    labels=['low', 'mid', 'high'])
# 10/20 low, 30/40 mid, 50 high
```

More Speed and Memory Habits

```
pd.read_csv('sales.csv', usecols=['region', 'price', 'qty'])  
# read only columns needed for analysis  
  
pd.read_csv('sales.csv', dtype={'region': 'category', 'qty': 'int32'})  
# set smaller dtypes while loading  
  
df['qty'] = pd.to_numeric(df['qty'], downcast='integer')  
# convert int64 to a smaller integer type when possible  
  
df.loc[df['price'] > 30, 'flag'] = 'high'  
# safe assignment; avoids chained assignment warnings (multiple  
consecutive operations)
```

* Using `.loc` is important because it updates `df` directly and avoids ambiguous chained assignment such as:

```
df[df["price"] > 30]["flag"] = "high" # unreliable
```

That chained expression may modify a temporary copy rather than the original DataFrame, potentially producing a `SettingWithCopyWarning`.

Cheat Sheet

```
df.groupby('category')['price'].transform('mean')  
# category mean repeated for every original row  
  
df.pivot_table(index='region', columns='category', values='price')  
# mean price by region and category  
  
pd.crosstab(df['category'], df['region'], normalize='index')  
# row-wise percentages  
  
df.assign(revenue=lambda d: d['price'] * d['qty'])  
# chainable new revenue column  
  
df['name'].str.extract(r'^(\w+)')  
# extract first word from each name  
  
df.sort_values('price').groupby('category').tail(1)  
# highest-price row in each category
```

Common Problems

► Chained indexing

- Avoid expressions such as `df[df['score'] > 80]['grade'] = 'A'`.
- They may modify a temporary copy rather than the original DataFrame.
- Prefer: `df.loc[df['score'] > 80, 'grade'] = 'A'`.

► Missing values in string filters

- String methods can encounter NaN values.
- For example: `df['name'].str.contains('Smith', na=False)`.
- The option `na=False` treats missing values as non-matches.

► Slow row-wise apply

- `apply(axis=1)` executes a Python function for each row and can be slow.
- Prefer vectorized operations when possible.
- Example: `df['tax'] = df['income'] * 0.20`.
- For conditions, consider
`np.where()` <https://numpy.org/devdocs/reference/generated/numpy.where.html>
`pd.cut()` <https://pandas.pydata.org/docs/reference/api/pandas.cut.html>.

► Ambiguous reshaping

- `pivot()` requires each row-column combination to be unique.
- If duplicates exist, use `pivot_table()` with an aggregation.
- Example: `df.pivot_table(index='year', columns='region', values='sales', aggfunc='mean')`.

► Hidden indexes after aggregation

- `groupby()` often places grouping variables in the index.
- Use `.reset_index()` when you want a regular tabular result.
- Example: `df.groupby('region')['sales'].mean().reset_index()`.

Assignment: FEMA Disaster Declarations

- ▶ Work in a **group of two**; however, **each student must submit the completed assignment individually through Canvas**.
- ▶ The assignment must be completed during the class period.
- ▶ This assignment emphasizes independent data exploration, problem-solving, and productive collaboration.
- ▶ Examine the dataset and its variables, develop relevant analytical questions, and apply the pandas functions and data-transformation techniques covered in class.
- ▶ Your analysis must go beyond producing tables: justify your choices, identify meaningful patterns, and explain what the results reveal about the types, locations, and frequency of major disasters.
- ▶ Dataset: **OpenFEMA Disaster Declarations Summaries**

<https://www.fema.gov/openfema-data-page/disaster-declarations-summaries-v2>

Assignment: Before You Begin

Before writing code, work with your partner to develop a clear analytical workflow.

1. Identify the variables needed for the analysis.
2. Break the assignment into a logical sequence of tasks.
3. Write pseudocode describing the required filtering, cleaning, grouping, aggregation, and visualization steps.
4. Decide how you will validate your results and avoid double-counting disasters.

Required Paper Submission

Write your workflow and pseudocode on paper and give it to me before the class period ends. This is a required part of the assignment!

Assignment: Expectations for Coding

No Vibe Coding

Do not generate or assemble code that you cannot explain. You must understand the purpose of every major step in your notebook.

- ▶ Wrestle with the problem, experiment with pandas, and do your best.
- ▶ When you encounter something you do not know, identify what you need to learn and work through it.
- ▶ Use documentation, class materials, examples, and discussion with your partner to resolve problems.
- ▶ Test your code incrementally rather than attempting to produce the entire analysis at once.
- ▶ Both partners should participate in developing, debugging, and interpreting the analysis.
- ▶ You should be prepared to explain your workflow, code, and conclusions.

Assignment: Data Preparation

Use pandas to inspect, clean, and transform the data. Retain records where:

- ▶ `declarationType` indicates a **major disaster**.
- ▶ `IA Program Declared` indicates that Individual Assistance was approved.

Inspect and document:

- ▶ Relevant variables and their data types.
- ▶ Missing values, if any, and how they are handled.
- ▶ Duplicate records, if any, and whether they should be removed.
- ▶ Any required string or date transformations.

Important Measurement Distinction

Clearly distinguish between a unique disaster, identified by `disasterNumber`, and the designated areas affected by that disaster. Count each appropriately and explain your method.

Assignment: Required Analysis

Analyze the frequency, location, type, and timing of major disasters:

- ▶ Identify the most common **disaster types**.
- ▶ Rank states by the number of **unique disasters**.
- ▶ Rank counties or designated areas by the number of unique disasters.
- ▶ For each disaster type, identify the states and designated areas most frequently affected.
- ▶ Examine how disaster declarations have changed over time.
- ▶ Develop and answer at least one additional analytical question of your own.

Use appropriate pandas operations, including filtering, `groupby()`, aggregation, sorting, missing-value handling, duplicate handling, and string or date transformations.

Assignment: Deliverables

Each student must submit a **Jupyter Notebook through Canvas**. The notebook must contain:

- ▶ Reproducible Python code that runs from beginning to end.
- ▶ Brief explanations of the workflow and analytical choices.
- ▶ Clearly labeled summary tables and visualizations.
- ▶ Concise interpretations of the main findings.
- ▶ An answer to the additional question developed by your group.
- ▶ A discussion of relevant data-quality or measurement limitations.

Before Leaving Class

Give me your group's paper workflow and pseudocode, and ensure that both students have individually submitted the completed notebook through Canvas.

Assignment: Final Conclusion

Conclude your notebook by answering:

What are the major disaster patterns in the United States? Identify the most common disaster types, the states and designated areas most frequently affected overall and by disaster type, and any important changes over time.

Your conclusion should:

- ▶ Refer to specific evidence from your analysis.
- ▶ Distinguish between counts of unique disasters and counts of affected areas.
- ▶ Explain the most important patterns rather than simply repeating values from tables.
- ▶ Acknowledge limitations that affect how the findings should be interpreted.

Assignment: Get started ...

```
# !pip install certifi  
import os  
import certifi  
os.environ["SSL_CERT_FILE"] = certifi.where()
```

```
import pandas as pd  
url = "https://www.fema.gov/api/open/v2/DisasterDeclarationsSummaries.  
      csv"  
df = pd.read_csv(url)  
df.head()
```

...last but not least

Dataset formats serve different workloads

Format	Structure	Good fit	Main tradeoff
CSV	Plain-text rows and columns	Spreadsheets and simple table exchange	No native types or nested data
JSON	One JSON object, possibly with metadata and nested records	Document-oriented and hierarchical data	Large documents may require full parsing
JSONA	One JSON array containing every record	Exporting a complete collection of records	Awkward to stream or append
JSONL	One independent JSON object per line	Logs, streaming, and record-wise processing	The complete file is not one JSON document
Parquet	Binary, typed, column-oriented storage	Large-scale analytics, selective column scans	Requires compatible software and is not human-readable

JSONA commonly means a JSON array of records; naming conventions vary.

CSV

```
id,name,score
1,Ana,95
2,Lee,88
```

JSON array (JSONA)

```
[
  {"id": 1, "name": "Ana", "score":
    95},
  {"id": 2, "name": "Lee", "score": 88}
]
```

JSON Lines (JSONL)

```
{"id": 1, "name": "Ana", "score": 95}
{"id": 2, "name": "Lee", "score": 88}
```

- ▶ **CSV**: simple tables and broad interoperability
- ▶ **JSON/JSONA**: nested fields and document structure
- ▶ **JSONL**: streaming, logs, and independent records
- ▶ **Parquet**: compact storage and fast analytical queries

Use a text format for inspection and interchange.
Use Parquet when analytical performance and storage efficiency matter.